

Bases de Dados

FORMAS NORMAIS – PARTE 2

©José Barateiro 2024
Adaptado de Silberschatz, Korth e Sudarschan

Forma Normal de Boyce-Codd

- Uma relação *R* está na forma BCNF em relação a um conjunto *F* de dependências funcionais se para todas as dependências funcionais em F^+ da forma

$$\alpha \rightarrow \beta$$

onde $\alpha \subseteq R$ e $\beta \subseteq R$, pelo menos uma das seguintes situações é válida:

  - $\alpha \rightarrow \beta$ é trivial (ou seja, $\beta \subseteq \alpha$)
  - α é uma superchave para *R*

Forma normal de Boyce-Codd (continuação)

- Exemplo de esquema que ***não está*** na forma BCNF:

  in_dep (*ID, name, salary, dept_name, building, budget*)

  porque :
  - *dept_name* → *building, budget*
    - verifica-se em *in_dep*
    - mas *dept_name* não é uma superchave

- Quando se decompõe *in_dept* em *instrutor* e *department*
  - *instrutor* está na BCNF
  - *departament* está na BCNF

Decompondo um esquema para BCNF

- Seja R um esquema que não está na BCNF. Seja $\alpha \rightarrow \beta$ a dependência funcional que causa uma violação do BCNF.
- Decompomos *R* em:
  - (α U β)
  - (*R* - (β - α))
- No emplo anterior *in_dep* ,
  - α = *dept_name*
  - β = *building, budget*
- Então, in_*dep* (*ID, name, salary, dept_name, building, budget)* deve ser decomposto em:
  - (α U β) = (*dept_name, building, budget*)
  - (*R* - (β - α)) = (*ID, name, dept_name, salary*)

Exemplo

- $R = (A, B, C)$
  $F = \{A \rightarrow B, B \rightarrow C)$

- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Decomposição sem perdas:

    $R_1 \cap R_2 = \{ B \}$ e $B \rightarrow BC$
  - Preservação da dependência

- $R_1 = (A, B)$, $R_2 = (A, C)$
  - Decomposição sem perdas:

    $R_1 \cap R_2 = \{ A \}$ e $A \rightarrow$ A B
  - Não preserva a dependência
    (não é possível verificar $B \rightarrow C$ sem calcular $R_1 \bowtie R_2$)

BCNF e preservação de dependência

- Nem sempre é possível atingir a BCNF com preservação da dependência
- Considere um esquema:

  dept_advisor(s_ID, i_ID, department_name)
- Com dependências funcionais:

  $i_ID \rightarrow dept_name$

  $s_ID, dept_name \rightarrow i_ID$
- *dept_advisor* não está na BCNF
  - *i_ID* não é uma superchave.
- Qualquer decomposição de *dept_advisor* não incluirá todos os atributos em

  $s_ID, dept_name \rightarrow i_ID$

Assim, não é possível garantir BCNF e preservação da dependência

Terceira forma normal

Um esquema de relação *R* está na **terceira forma normal (3FN)** se para todos:

$$\alpha \rightarrow \beta \text{ em } F^{+}$$

pelo menos uma das seguintes situações é válida:

- $\alpha \rightarrow \beta$ é trivial (ou seja, $\beta \in \alpha$)
- α é uma superchave para *R*
- Cada atributo *A* em $\beta - \alpha$ está contido numa chave candidata para *R*.

  (**NOTA** *:* cada atributo pode estar numa chave candidata diferente)

- Se uma relação está na BCNF, ela está na 3FN (já que na BCNF uma das duas primeiras condições acima deve ser válida).
- A terceira condição é um relaxamento da BCNF para garantir a preservação da dependência.

Exemplo 3FN

- Considere um esquema:

  dept_advisor (s_ID , i_ID , dept_name)

- Com dependências funcionais:

  $i_ID \rightarrow dept_name$

  $s_ID, dept_name \rightarrow i_ID$

- Duas chaves candidatas = { *s_ID , dept_name* }, { *s_ID , i_ID* }

- Já vimos anteriormente que *dept_advisor* não está na BCNF

- *N*o entanto, está em 3FN, pois
  - *s_ID , dept_name* é uma superchave
  - $i_ID \rightarrow dept_name$ *e i_ID* NÃO é uma superchave , mas:
    - { *dept_name*} – {*i_ID* } = {*dept_name* } e
    - *dept_name* está contido numa chave candidata

Redundância na 3NF

Considere o esquema R abaixo, que está na 3FN

- $R = (J, K, L)$
- $F = \{ JK \rightarrow L, L \rightarrow K \}$
- E a instância:

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

- Repetição de informação
- É necessário usar valores nulos (por exemplo, para representar o relacionamento l_2, k_2 onde não há valor correspondente para J)

Comparação de BCNF e 3NF

- Vantagens do 3FN sobre a BCNF. É sempre possível obter um desenho na 3FN sem sacrificar a ausência de perdas e a preservação de dependência.

  Desvantagens da 3FN.
  - Pode ser necessário usar valores nulos para representar algumas das possíveis associações entre dados.
  - Pode existir o problema da repetição.

Objetivos da Normalização

- Seja *R* um esquema de relação com um conjunto *F* de dependências funcionais.
- Decida se um esquema de relação *R* está em "boa" forma.
- No caso de um esquema de relação *R* não estar em "boa" forma, é necessário decompô-lo num conjunto de esquemas de relação { R_1, R_2, ..., R_n } tais que:
  - Cada esquema de relação está em "boa" forma
  - A decomposição é uma decomposição sem perdas
  - De preferência, a decomposição deve preservar a dependência.

Fecho de um conjunto de dependências funcionais

- Dado um conjunto *F* de dependências funcionais, há outras dependências funcionais que são logicamente implícitas por *F* .
  - Se $A \rightarrow B$ e $B \rightarrow C$, então podemos inferir que $A \rightarrow$ C
  - etc.
- O conjunto de **todas** as dependências funcionais logicamente implícitas por *F* é o **fecho** de *F* .

  Denotamos o *fecho* de *F* por F^+ .

Fecho de um conjunto de dependências funcionais

Podemos calcular F^+, o fecho de F, aplicando repetidamente **os Axiomas de Armstrong :**

- **Regra reflexiva:** se $\beta \subseteq \alpha$, então $\alpha \rightarrow \beta$
- **Regra de aumento :** se $\alpha \rightarrow \beta$, então $\gamma\ \alpha \rightarrow \gamma\ \beta$
- **Regra de transitividade :** se $\alpha \rightarrow \beta$, e $\beta \rightarrow \gamma$, então $\alpha \rightarrow \gamma$

Essas regras são

- **Corretas** -- geram apenas dependências funcionais válidas
- **Completas** — geram todas as dependências funcionais válidas

Exemplo de F^+

- $R = (A, B, C, G, H, I)$

  $F = \{ A \rightarrow B$

  $A \rightarrow C$

  $CG \rightarrow H$

  $CG \rightarrow I$

  $B \rightarrow H\}$

- Alguns membros do F^+
  - $A \rightarrow H$
    - por transitividade de $A \rightarrow B$ *e* $B \rightarrow H$
  - $AG \rightarrow I$
    - aumentando $A \rightarrow C$ com G, para obter $AG \rightarrow CG$ e então transitividade com $CG \rightarrow I$
  - $CG \rightarrow HI$
    - aumentando $CG \rightarrow I$ para inferir $CG \rightarrow CGI$,

      e aumento de $CG \rightarrow H$ para inferir $CGI \rightarrow HI$,

      e então transitividade

Fecho de Dependências Funcionais (Cont.)

Regras adicionais:

- **União** : Se $\alpha \rightarrow \beta$ e $\alpha \rightarrow \gamma$, então $\alpha \rightarrow \beta\gamma$
- **Decomposição** : Se $\alpha \rightarrow \beta\gamma$, então $\alpha \rightarrow \beta$ e $\alpha \rightarrow \gamma$
- **Pseudotransitividade** : Se $\alpha \rightarrow \beta$ e $\gamma\ \beta \rightarrow \delta$, então $\alpha\ \gamma \rightarrow \delta$.

As regras acima podem ser inferidas dos axiomas de Armstrong .

Procedimento para calcular F^+

Para calcular o fecho de um conjunto de dependências funcionais F:

$F^+ = F$
repeat
  for each dependência funcional f em F^+
    aplicar regras de reflexividade e aumento em f
    adicione as dependências funcionais resultantes a F^+
  for each par de dependências funcionais f_1 e f_2 em F^+
    if f_1 e f_2 podem ser combinados usando transitividade
      then adicione a dependência funcional resultante a F^+
until F^+ não mude mais

Fecho de conjuntos de atributos

- Dado um conjunto de atributos α, define-se o ***fecho*** de α **sob** *F* (denotado como α^+) como o conjunto de atributos que são funcionalmente determinados por α em *F*
- Algoritmo para calcular α^+ , o fecho de α sob *F*

```
resultado := α ;
    while (mudanças no resultado ) do
        for each β → γ in F do
            begin
                if β ⊆ resultado then resultado := resultado ∪ γ
            end
```

Exemplo de fecho de conjunto de atributos

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
  $A \rightarrow C$
  $CG \rightarrow H$
  $CG \rightarrow I$
  $B \rightarrow H\}$
- $(AG)^+$
  1. *resultado = AG*
  2. *resultado = ABG (A → B)*
  3. *resultado = ABCG (A → C)*
  4. *resultado = ABCGH (CG → H)*
  5. *resultado = ABCGHI (CG → I)*
- *AG* é uma chave candidata?
  1. AG é uma superchave?
     1. $AG \rightarrow R? == (AG)^+ \supseteq R$
  2. Algum subconjunto de AG é uma superchave ?
     1. $A \rightarrow R ? == (A)^+ \supseteq R$
     2. $G \rightarrow R ? == (G)^+ \supseteq R$
     3. Em geral: verifique cada subconjunto de tamanho *n-1*

Usos do Fecho de Atributos

- Existem vários usos do algoritmo de fecho de atributos:
- Testar superchave :
  - Para testar se α é uma superchave , calculamos α^+ e verificamos se α^+ contém todos os atributos de *R.*
- Testar dependências funcionais
  - Para verificar se uma dependência funcional $\alpha \rightarrow \beta$ é válida (i.e., está em F^+), basta verificar se $\beta \subseteq \alpha^+$.
  - Ou seja, calculamos α^+ usando o fecho de atributo e então verificamos se contém β.
- Calcular o fecho de F
  - Para cada $\gamma \subseteq R$, encontramos o fecho γ^+ , e para cada $S \subseteq \gamma^+$, produzimos uma dependência funcional $\gamma \rightarrow S$.

Cobertura canónica

- Considere um conjunto de dependências funcionais *F*. Sempre que houver uma atualização na relação, o SGBD deve garantir que a atualização não viola nenhuma dependência funcional; ou seja, todas as dependências funcionais em *F* são satisfeitas no novo estado da BD.
- Se uma atualização violar qualquer dependência funcional no conjunto *F,* o sistema deverá reverter a atualização.
- Podemos reduzir o esforço gasto na verificação, usando um conjunto simplificado de dependências funcionais que tenham o mesmo fecho que o conjunto fornecido.
- Este conjunto simplificado é denominado **cobertura canónica**
- Para definir a cobertura canónica, devemos primeiro definir **os atributos estranhos.**
  - Um atributo de uma dependência funcional em *F* é **estranho** se pudermos removê-lo sem alterar F^+

Atributos estranhos

- Remover um atributo do lado esquerdo de uma dependência funcional transforma-a numa restrição mais forte.
  - Por exemplo, se tivermos AB $\rightarrow$ C e removermos B, obtemos o resultado possivelmente mais forte A $\rightarrow$ C. É mais forte porque A $\rightarrow$ C implica logicamente AB $\rightarrow$ C, mas AB $\rightarrow$ C não implica logicamente A $\rightarrow$ C por si só .
- Dependendo do conjunto F de dependências funcionais, decidimos se é possível remover B de AB $\rightarrow$ C com segurança.
  - Por exemplo, suponha que
  - F = {AB $\rightarrow$ C, A $\rightarrow$ D, D $\rightarrow$ C}
  - Então podemos mostrar que F implica logicamente A $\rightarrow$ C, tornando o atributo B estranho em AB $\rightarrow$ C.

Atributos estranhos (continuação)

- Remover um atributo do lado direito de uma dependência funcional transforma-a numa restrição mais fraca.
  - Por exemplo, se tivermos AB $\rightarrow$ CD e removermos C, obteremos a dependência mais fraca AB $\rightarrow$ D. É mais fraca porque, usando apenas AB $\rightarrow$ D, não podemos inferir AB $\rightarrow$ C.
- Mas, dependendo do conjunto F de dependências funcionais, decidimos se podemos remover C de AB $\rightarrow$ CD com segurança.
  - Por exemplo, suponha que

  F = { AB $\rightarrow$ CD, A $\rightarrow$ C}
  - Então podemos mostrar que mesmo depois de substituir AB $\rightarrow$ CD por AB $\rightarrow$ D, ainda podemos inferir AB $\rightarrow$ C e, portanto, AB $\rightarrow$ CD.

Atributos estranhos

- Um atributo de uma dependência funcional em *F* é **estranho** se pudermos removê-lo sem alterar F^+
- Considere um conjunto *F* de dependências funcionais e a dependência funcional $\alpha \rightarrow \beta$ em *F* .
  - **Remover do lado esquerdo** : O atributo A é **estranho** em α se
    - $A \in \alpha$ e
    - *F* implica logicamente $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
  - **Remover do lado direito** : O atributo *A* é **estranho** em β se
    - $A \in \beta$ e
    - O conjunto de dependências funcionais

      $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ implica logicamente *F.*

Nota: a implicação na direção oposta é trivial em cada um dos casos acima, uma vez que uma dependência funcional "mais forte" implica sempre uma mais fraca

Testar se um atributo é estranho

- Seja *R* um esquema de relação e seja *F* um conjunto de dependências funcionais que se verificam em *R.* Considere um atributo na dependência funcional $\alpha \rightarrow \beta$.
- Para testar se o atributo $A \in \beta$ é estranho em β
  - Considere o conjunto:
    $F' = (F - \{ \alpha \rightarrow \beta \}) \cup \{\alpha \rightarrow (\beta - A)\}$,
  - verifique se α^+ contém *A;* se contiver, *A* é estranho em β
- Para testar se o atributo $A \in \alpha$ é estranho em α
  - Seja $\gamma = \alpha - \{A\}$. Verifique se $\gamma \rightarrow \beta$ pode ser inferido de *F.*
    - Calcule γ^+ usando as dependências em *F*
    - Se γ^+ inclui todos os atributos em β então , *A* é estranho em α

Exemplos de atributos estranhos

- Seja $F = \{ AB \rightarrow CD , A \rightarrow E, E \rightarrow C \}$
- Para verificar se C é estranho em $AB \rightarrow CD$:
  - Calcule o fecho de AB sob $F' = \{ AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$
  - O fecho de AB é *ABCDE,* que inclui *CD*
  - Isto implica que *C* é estranho

Cobertura canónica

A **cobertura canónica** para F é um conjunto de dependências F_c tal que

- F implica logicamente todas as dependências em F_c, e
- F_c implica logicamente todas as dependências em F, e
- Nenhuma dependência funcional em F_c contém um atributo estranho e
- Cada lado esquerdo de uma dependência funcional em F_c é único. Ou seja, não há duas dependências em F_c
  - $\alpha_1 \rightarrow \beta_1$ e $\alpha_2 \rightarrow \beta_2$ tal que
  - $\alpha_1 = \alpha_2$

Cobertura canónica

- Para calcular uma cobertura canónica para *F* :

repeat

Use a regra de união para substituir quaisquer dependências em *F* do tipo

$$\alpha_1 \rightarrow \beta_1 \text{ e } \alpha_1 \rightarrow \beta_2 \text{ com } \alpha_1 \rightarrow \beta_1 \beta_2$$

Encontre uma dependência funcional $\alpha \rightarrow \beta$ em F_c com um atributo estranho em α ou em β

Se encontrar um atributo estranho, exclua-o de $\alpha \rightarrow \beta$

até (F_c não mudar)

Nota: A regra da União pode se tornar aplicável após alguns atributos estranhos terem sido excluídos, portanto, ela deve ser reaplicada

Exemplo: cálculo da cobertura canónica

- $R = (A, B, C)$
  $F = \{A \rightarrow BC , B \rightarrow C , A \rightarrow B, AB \rightarrow C \}$
- Combine $A \rightarrow BC$ e $A \rightarrow B$ para $A \rightarrow BC$
  - O conjunto agora é $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- A é estranho em $AB \rightarrow C$?
  - Verifique se o resultado da exclusão de A de $AB \rightarrow C$ é implícito nas outras dependências
    - Sim: na verdade, $B \rightarrow C$ já está presente!
  - O conjunto agora é $\{A \rightarrow BC, B \rightarrow C \}$
- C é estranho em $A \rightarrow BC$
  - Verifique se $A \rightarrow C$ é logicamente implicado por $A \rightarrow B$ e as outras dependências
    - Sim *:* usando transitividade em $A \rightarrow B$ *e* $B \rightarrow$ C.
      - Pode usar-se o fecho do atributo A em casos mais complexos
- A cobertura canónica é: $A \rightarrow B , B \rightarrow C$

Formas Normais

VERSÃO SIMPLIFICADA

Primeira forma normal (1FN)

- Um domínio é **atómico** se os seus elementos são considerados unidades indivisíveis
  - Exemplos de domínios não atómicos:
    - Atributos multiplo valor (números de telefone)
    - Atributos com informação de vários conceitos (e.g., C1.25 – Edifício C, piso 1, sala 25)
- Um esquema relacional R está na **primeira forma normal (1FN)** se os domínios de todos os atributos de R são atómicos

Primeira forma normal (continuação)

Atomicidade é na verdade uma propriedade de como os elementos do domínio são usados.

- Exemplo: Strings normalmente seriam consideradas indivisíveis
- Suponha os nomes dos clientes são usados para colocar um título, e.g., “Engª Maria”, “Prof. António”
- Se a primeira palavra for usada para extrair o título, o domínio não é atómico.
- Esta abordagem é muito má ideia: leva à necessidade de codificação no lado da aplicação.

Segunda forma normal (2FN)

- Um esquema relacional R está na **segunda forma normal (2FN)** se está na 1FN e **cada atributo não chave depende completamente dos atributos da chave**.
  - Se estiver na 1FN e chave simples, está na 2FN
  - Não avalia dependências entre atributos de diferentes chaves
  - Não avalia dependências entre atributos não chave

  - NOTA: ser completamente dependente, significa que não depende de parte da chave. Por exemplo, na dependência AB - > C, diz-se que C depende completamente de AB, se não se verifica que A -> C nem B -> C

Terceira forma normal (3FN)

Um esquema de relação *R* está na **terceira forma normal (3FN)** se para todas:

$$\alpha \rightarrow \beta \text{ em } F^+$$

onde $\alpha \subseteq R$ e $\beta \subseteq R$, pelo menos uma das seguintes situações é válida:

- $\alpha \rightarrow \beta$ é trivial (ou seja, $\beta \in \alpha$)
- α é uma superchave para *R*
- Cada atributo *A* em $\beta - \alpha$ está contido numa chave candidata para *R*.
  (**NOTA** *:* cada atributo pode estar numa chave candidata diferente)

- Um esquema relacional R está na **terceira forma normal (3FN)** se está na 2FN e **não existem dependências entre os atributos não chave**.

  - Não avalia dependências entre atributos de diferentes chaves

Boyce Codd Normal Form (BCNF)

- Uma relação *R* está na forma BCNF em relação a um conjunto *F* de dependências funcionais se para todas as dependências funcionais em F^+ da forma

$$\alpha \rightarrow \beta$$

onde $\alpha \subseteq R$ e $\beta \subseteq R$, pelo menos uma das seguintes situações é válida:

  - $\alpha \rightarrow \beta$ é trivial (ou seja, $\beta \subseteq \alpha$)
  - α é uma superchave para *R*

- Um esquema relacional R está na **BCNF** se em todas as dependências funcionais o determinante é uma superchave (ou dependências triviais).
- Um esquema relacional R está na **BCNF** se está na 3FN e **não existem dependências entre subconjuntos dos atributos das chaves**.

Boyce Codd Normal Form (BCNF)

- Exemplo revisitado; Considere um esquema:

  dept_advisor (s_ID , i_ID , dept_name)

  Com dependências funcionais:

  $i_ID \rightarrow dept_name$

  $s_ID, dept_name \rightarrow i_ID$

  Duas chaves candidatas = { *s_ID , dept_name* }, { *s_ID , i_ID* }

- Assumimos que está na 1FN (todos os domínios atómicos)
- Está na 2FN, os atributos não-chave dependem completamente das chaves (não há atributos não-chave)
- Está na 3FN, pois não existem dependências entre atributos não-chave (não há atributos não-chave)
- Não está na BCFN pois existem dependências entre subconjuntos dos atributos das chaves: $i_ID \rightarrow dept_name$